

Guia de Implementação Oficial: Spring Boot + MariaDB (Fluxo Marlos) - V2

Este documento apresenta o passo a passo definitivo e completo para implementar a arquitetura de software exatamente como desenhada no fluxograma oficial do professor, refletindo a versão final e validada do seu código com as anotações do Lombok e as correções do Service.

Passo 1: Criação do Banco de Dados no MariaDB

Abra o seu terminal ou cliente de banco de dados (HeidiSQL, DBeaver, etc.) e crie o banco de dados:

```
CREATE DATABASE logindb CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

Passo 2: Configuração das Propriedades (application.properties)

Configure o arquivo src/main/resources/application.properties para estabelecer a comunicação direta com o banco logindb:

```
spring.application.name=thymeleaf  
# Conexão MariaDB  
spring.datasource.url=jdbc:mariadb://localhost:3306/logindb  
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver  
spring.datasource.username=root  
spring.datasource.password=sua_senha_aqui  
  
# Configurações de Criação Automática e Dialeto  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true  
server.port=8080
```

Passo 3: Criação da ENTITY (Mapeamento do Banco)

No pacote com.senai.thymeleaf.entities, a classe UsuarioEntity.java utilizando o Lombok:

```
package com.senai.thymeleaf.entities;
```

```

import jakarta.persistence.*;
import lombok.Data;

@Data
@Entity
@Table(name = "tb_usuarios")
public class UsuarioEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    @Column(name = "email", unique = true, nullable = false)
    private String email;

    @Column(name = "senha", nullable = false)
    private String senha;

    public UsuarioEntity() {
    }
}

```

Passo 4: Criação do REPOSITORY (Consultas SQL Dinâmicas)

No pacote com.senai.thymeleaf.repositories, a interface UsuarioRepository.java:

```

package com.senai.thymeleaf.repositories;

import com.senai.thymeleaf.entities.UsuarioEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UsuarioRepository extends
    JpaRepository<UsuarioEntity, Long> {
    Optional<UsuarioEntity> findByEmail(String email);
}

```

Passo 5: Criação do DTO (Data Transfer Object)

No pacote com.senai.thymeleaf.dtos, a classe LoginDto.java utilizando o Lombok:

```

package com.senai.thymeleaf.dtos;

import lombok.Data;

@Data
public class LoginDto {
    private String email;
    private String senha;
}

```

Passo 6: Criação do SERVICE (Regra de Autenticação)

No pacote `com.senai.thymeleaf.services`, a classe `LoginService.java` devidamente anotada:

```

package com.senai.thymeleaf.services;

import com.senai.thymeleaf.dtos.LoginDto;
import com.senai.thymeleaf.entities.UsuarioEntity;
import com.senai.thymeleaf.repositories.UsuarioRepository;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class LoginService {

    private final UsuarioRepository repository;

    public LoginService(UsuarioRepository repository) {
        this.repository = repository;
    }

    public Optional<UsuarioEntity> autenticar(LoginDto login) {
        //Buscar no banco através do Repository utilizando o email
        Optional<UsuarioEntity> usuarioOpt =
        repository.findByEmail(login.getEmail());

        //Se existir valida regra
        if (usuarioOpt.isPresent()) {
            UsuarioEntity usuario = usuarioOpt.get();
            if (usuario.getSenha().equals(login.getSenha())) {
                return Optional.of(usuario);
            }
        }
        return Optional.empty(); //Falha na autenticação
    }
}

```

```
}  
}
```

Passo 7: Atualização do CONTROLLER (Gerenciamento do Fluxo)

No pacote `com.senai.thymeleaf.controllers`, o seu `HomeController.java`:

```
package com.senai.thymeleaf.controllers;  
  
import com.senai.thymeleaf.dtos.LoginDto;  
import com.senai.thymeleaf.entities.UsuarioEntity;  
import com.senai.thymeleaf.services.LoginService;  
import jakarta.servlet.http.HttpSession;  
import org.springframework.stereotype.Controller;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.ui.Model;  
import org.springframework.web.bind.annotation.PostMapping;  
  
import java.util.Optional;  
  
@Controller  
public class HomeController {  
  
    private final LoginService loginService;  
  
    // Injeção de dependência do Service oficial  
    public HomeController(LoginService loginService) {  
        this.loginService = loginService;  
    }  
  
    @GetMapping("/")  
    public String inicio(HttpSession session, Model model) {  
        model.addAttribute("mensagem", "Bem-vindo ao Thymeleaf!");  
        String usuario = (String)  
session.getAttribute("usuarioLogado");  
        if (usuario == null) {  
            return "redirect:/login";  
        }  
  
        model.addAttribute("usuario", usuario);  
        return "home";  
    }  
  
    @GetMapping("/login")
```

```

    public String telaLogin() {
        return "login";
    }

    @PostMapping("/login")
    public String fazerLogin(LoginDto loginDto, HttpSession session,
Model model) {
        //Aciona serviço passando pelo Dto
        Optional<UsuarioEntity> usuarioAutenticacao =
loginService.autenticar(loginDto);

        if (usuarioAutenticacao.isPresent()) {
            //Guarda na sessão e faz o redirect
            session.setAttribute("usuarioLogado",
usuarioAutenticacao.get().getEmail());
            return "redirect:/";
        }
        //Se falhar retorna erro
        model.addAttribute("erro", "Email ou Senha inválidos");
        return "login";
    }

    @GetMapping("/logout")
    public String logout(HttpSession session) {
        session.invalidate();
        return "redirect:/login";
    }
}

```

Passo 8: Inserção do Usuário no MariaDB

Para criar o usuário de testes inicial (Admin), execute o comando abaixo no seu cliente de banco de dados:

```

INSERT INTO tb_usuarios (email, senha) VALUES ('admin@senai.com',
'123');

```